REPORT

Complementing a Strand of DNA

by

Swetali Smruti Sikhya

Student

B.Sc. Biology | Aspiring Computational Biologist

1. Problem Overview

In DNA strings, the symbols 'A' and 'T' are complements of each other, as are 'C' and 'G':

Table 1. DNA base complement pairs.

Nucleotide	Complement
A	T
T	A
C	G
G	C

The reverse complement of a DNA string s is the string s^c, formed by reversing the symbols of s and then taking the complement of each symbol. Given a DNA string, the task is to generate its reverse complement by:

- Replacing each nucleotide with its complement.
- Reversing the resulting string.

Given: A DNA string s of length at most 1000 bp.

Return: The reverse complement s^c of s.

1.1. Sample Dataset

```
AAAACCCGGT
```

1.2. Sample Output

```
ACCGGGTTTT
```

2. Approach

Once we know the order of bases on one strand, we can immediately deduce the sequence of bases on the complementary strand, since each base bonds with a fixed partner (A with T, C with G). Because the two strands of DNA run in opposite directions, the complementary strand must also be reversed to read it in the standard 5' to 3' direction.

The steps are:

1. Read the DNA string s from the input file.
2. Create a lookup table (dictionary) that stores each base's complementary pair.
3. Traverse s and replace each nucleotide with its complement, building a complemented string.
4. Reverse the complemented string to get the reverse complement.
5. Output the resulting reverse complement s^c to a file.

3. Algorithm

ReverseComplement(s):

```
1. Create a lookup table: {'A':'T', 'T':'A', 'C':'G', 'G':'C'}
2. Initialize an empty string: complement <- ""
3. FOR each base b in s:
4.     complement <- complement + lookup_table[b]
5. reverse_complement <- reverse(complement)
6. RETURN reverse_complement
```

In words:

- Build a small lookup table mapping each base to its complement.
- Go through the DNA string s one base at a time, appending the complement of each base to a new string.
- Once the full complemented string is built, reverse it end-to-end.
- The reversed, complemented string is the answer.

In Python, the complement step is expressed with a dictionary lookup inside a loop (or list comprehension), and the reversal step uses Python's slice notation [::-1], which reverses a string in a single, efficient operation.

4. Complexity Analysis

- Time Complexity: O(n) — traversing the DNA string to build the complement takes O(n), and reversing the string takes another O(n); the two combine to O(n) overall, where n is the length of s.
- Space Complexity: O(n), since a new string of the same length as s is created to hold the complemented (and later reversed) sequence.

Given the constraint ($n \leq 1000$ bp), this problem runs essentially instantly, even with the simplest implementation.

5. Python Implementation

Below is the finalized implementation. The logic is organized into four small, well-documented functions rather than one flat script, which makes it easier for any reader to follow: each function has a single, clearly named responsibility, and a main() function ties the steps together.

```
"""
Rosalind Problem: Complementing a Strand of DNA
-----------------------------------------------
Task:
    Given a DNA string 's' (length at most 1000 bp), produce its
    reverse complement 's^c'. The reverse complement is formed by:
        1. Replacing each base with its complement
           (A <-> T, C <-> G).
        2. Reversing the resulting string.

Input:
    A plain text file named 'rosalind_revc.txt' containing a single
    line with the DNA string.
```

```
Output:
    A plain text file named 'rosalind_revc_output.txt' containing a
    single line with the reverse complement string.
"""

# ---------------------------------------------------------------------
# File names used for input and output.
# Keeping them as named constants (instead of "magic strings" scattered
# through the code) makes the script easier to read and update.
# ---------------------------------------------------------------------
INPUT_FILE = "rosalind_revc.txt"
OUTPUT_FILE = "rosalind_revc_output.txt"

# ---------------------------------------------------------------------
# Lookup table for complementary DNA bases.
# A pairs with T, and C pairs with G (Watson-Crick base pairing).
# ---------------------------------------------------------------------
COMPLEMENT = {
    "A": "T",
    "T": "A",
    "C": "G",
    "G": "C",
}

# ---------------------------------------------------------------------
# FUNCTIONS USED IN THIS SCRIPT (4 custom functions total):
#   1. reverse_complement(dna_string)      -> computes the reverse complement
#   2. read_dna_sequence(file_path)        -> reads the DNA string from a file
#   3. write_rna_sequence(file_path, seq)  -> writes the result to a file
#   4. main()                              -> runs steps 1-3 in order
#
# Each one is defined with "def" below and marked with a comment
# "# FUNCTION:" right above it, so they're easy to spot.
# ---------------------------------------------------------------------


# FUNCTION: reverse_complement
def reverse_complement(dna_string):
    """
    Compute the reverse complement of a DNA sequence.

    Every base is first replaced by its complementary base
    (A <-> T, C <-> G) using the COMPLEMENT lookup table, and the
    resulting complemented string is then reversed.

    Parameters
    ----------
    dna_string : str
        A DNA sequence made up of the letters A, C, G, and T.

    Returns
    -------
    str
        The reverse complement of the input DNA sequence.

    Example
    -------
    >>> reverse_complement("AAAACCCGGT")
    'ACCGGGTTTT'
    """
    # Step 1: replace each base with its complement.
    complemented_bases = [COMPLEMENT[base] for base in dna_string]
    complement_string = "".join(complemented_bases)

    # Step 2: reverse the complemented string.
    return complement_string[::-1]
```

```
# FUNCTION: read_dna_sequence
def read_dna_sequence(file_path):
    """
    Read a DNA sequence from a text file.

    The file is expected to contain a single line with the DNA
    string. Leading/trailing whitespace (including the newline at
    the end of the file) is removed.

    Parameters
    ----------
    file_path : str
        Path to the input file containing the DNA sequence.

    Returns
    -------
    str
        The DNA sequence as a clean string (no surrounding whitespace).
    """
    with open(file_path, "r") as input_file:
        return input_file.read().strip()


# FUNCTION: write_rna_sequence
def write_rna_sequence(file_path, sequence):
    """
    Write a sequence to a text file, followed by a newline.

    Parameters
    ----------
    file_path : str
        Path to the output file to create (or overwrite).
    sequence : str
        The sequence to write to the file.
    """
    with open(file_path, "w") as output_file:
        output_file.write(sequence + "\n")


# FUNCTION: main
def main():
    """
    Run the full workflow:
    1. Read the DNA sequence from INPUT_FILE.
    2. Compute its reverse complement.
    3. Write the reverse complement to OUTPUT_FILE.
    4. Print the result to the screen as well, for a quick check.
    """
    dna_string = read_dna_sequence(INPUT_FILE)
    result = reverse_complement(dna_string)
    write_rna_sequence(OUTPUT_FILE, result)

    print(f"Reverse complement written to '{OUTPUT_FILE}':")
    print(result)


# The "if __name__ == '__main__':" guard means main() only runs when
# this file is executed directly (e.g. `python rosalind_revc_solution.py`),
# not when it's imported as a module into another script.
if __name__ == "__main__":
    main()
```

6. Result

The implementation was verified against the official Rosalind sample dataset:

6.1. Input

```
AAAACCCGGT
```

6.2. Step-by-Step Execution

Table 2. Step-by-step reverse complement computation for the sample dataset.

Original Base	Complement
A	T
A	T
A	T
A	T
C	G
C	G
C	G
G	C
G	C
T	A

6.3. Complement String

```
TTTTGGGCCA
```

6.4. After Reversing (Output)

```
ACCGGGTTTT
```

This matches the official Rosalind sample output exactly, confirming the correctness of the implementation. The actual dataset submitted to and accepted by Rosalind is unique to each user's account; that personal dataset and its corresponding output are therefore omitted from this report and are not included in the accompanying repository.

7. Skills Demonstrated

This introductory problem provided practice in several fundamental programming and bioinformatics concepts.

Bioinformatics

- DNA sequence manipulation
- Understanding nucleotide complementarity and base pairing
- Bioinformatics fundamentals

Python Programming

- Functions
- Dictionary usage (lookup tables)
- String manipulation and reverse indexing ([::-1])
- Loops and list comprehensions
- File input/output

Problem-Solving Skills

- Data transformation
- Algorithm design
- Computational thinking
- Efficient sequence processing

8. Conclusion

The Complementing a Strand of DNA problem is a fundamental bioinformatics exercise that introduces biological sequence processing using Python. The solution uses a dictionary-based mapping to generate complementary nucleotides and then reverses the sequence to obtain the reverse complement.

The algorithm runs in O(n) time and O(n) space, making it efficient even for large DNA sequences. This exercise strengthens both programming and bioinformatics concepts and serves as an excellent introduction to computational biology.